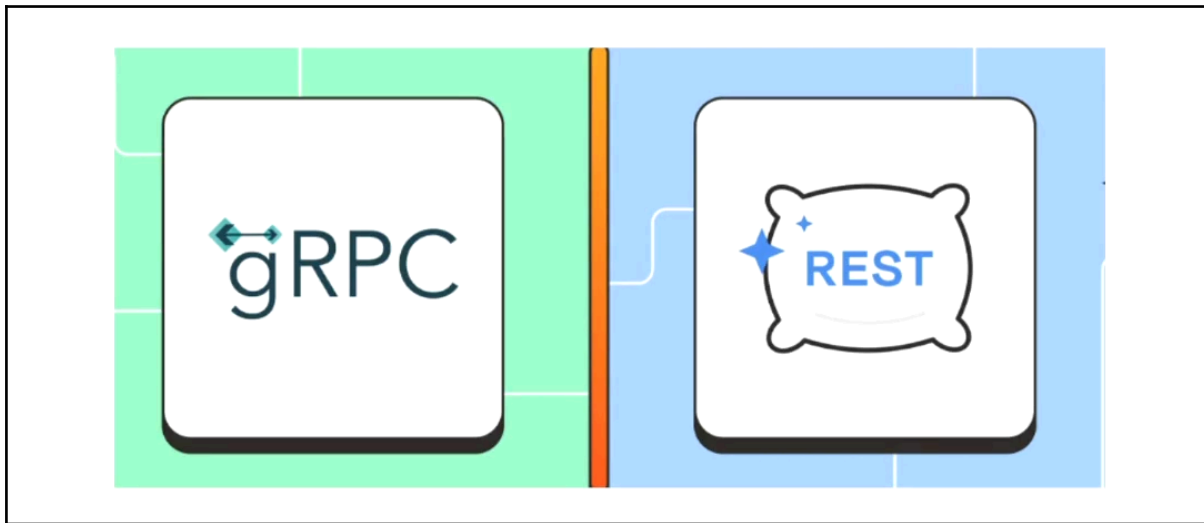


Section 3: System Design Fundamentals

Modern API Protocols - Beyond REST (gRPC, GraphQL)

While REST has been the dominant API standard for many years, newer approaches like gRPC, GraphQL address some of its limitations. In this session, we will explore why we need alternatives to REST, how gRPC and GraphQL works, their key advantages, and when to use each one.



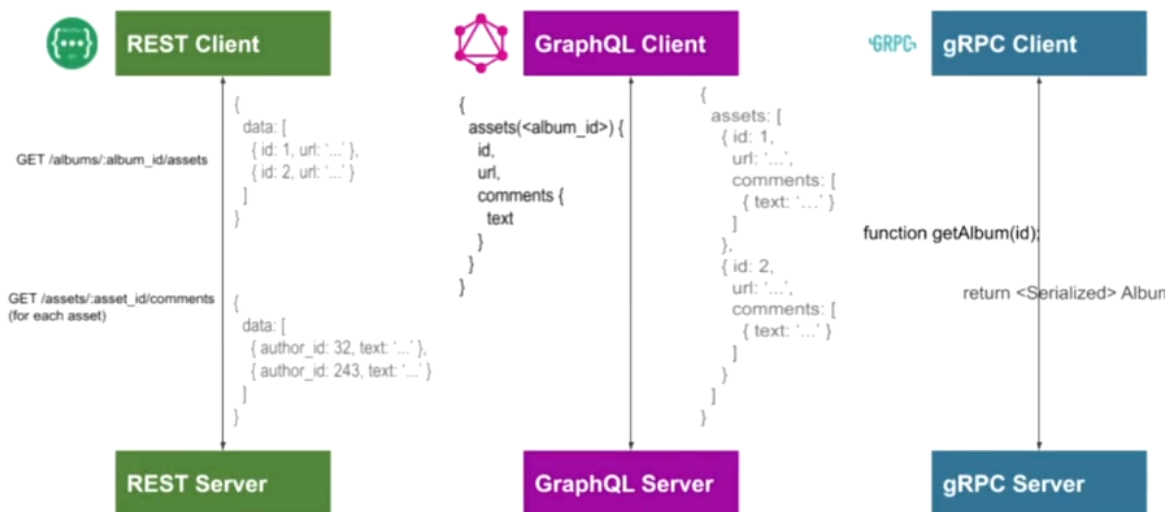
Why Do We Need More Than REST?

REST APIs has been the standard for building APIs for years, however it comes with some certain limitations that can impact performance and flexibility in modern applications. Some of these limitations are:

- **Overfetching & Underfetching:** The client often receives either too much of unnecessary data or too little data which is not enough for them, and thus requiring extra requests to be sent to the server. For example, fetching user details might return an entire profile of the user when you only need a name and email.
- **Higher Latency:** REST APIs often require multiple round trips to fetch related data. If a client needs a user's details along with their recent transactions, it may have to make multiple requests, which will increase the response time for the client.
- **No Real-Time Communication:** REST is not optimized for real-time communications. It relies on polling, where the clients repeatedly send requests to check for updates, which is inefficient and consumes extra resources.

To address these limitations, modern solutions like gRPC and GraphQL have emerged. gRPC improves performance and efficiency, especially in microservices, while GraphQL provides flexible data fetching, allowing clients to request exactly what they need.

Introducing gRPC & GraphQL

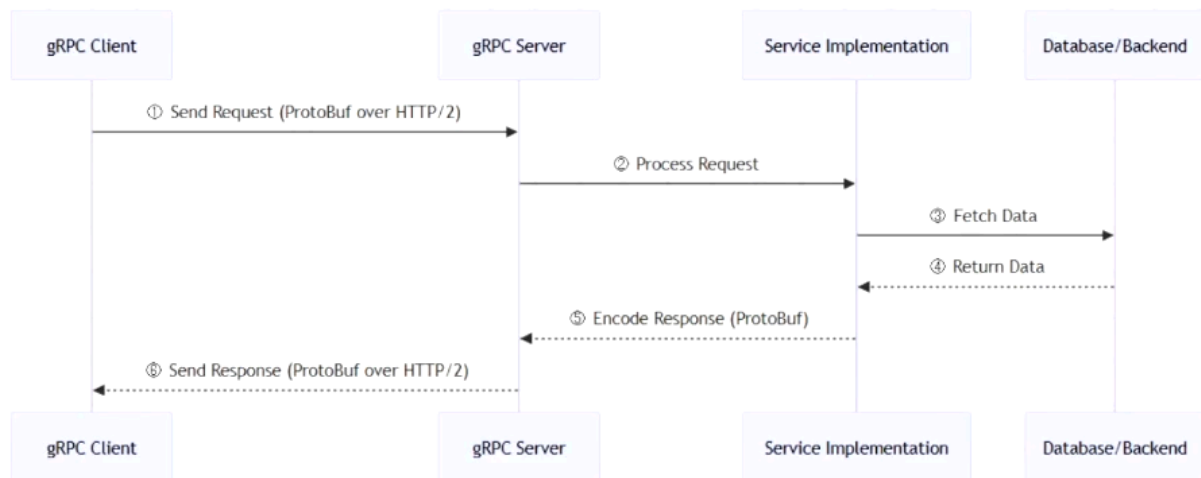


gRPC & GraphQL are two modern alternatives to REST.

- **gRPC:** It is a high performance API framework that uses protocol buffers (proto-buffs), instead of JSON, making it faster and more efficient. It is optimized for microservices, and it supports the real-time streaming and communication over HTTP/2, which makes it ideal for low latency, high performance in our applications.
- **GraphQL:** It is a query language that gives clients precise control over the data they are fetching. Instead of receiving a fixed response in REST, clients can request only the fields they need, which reduces overfetching and improves the performance for frontend applications.

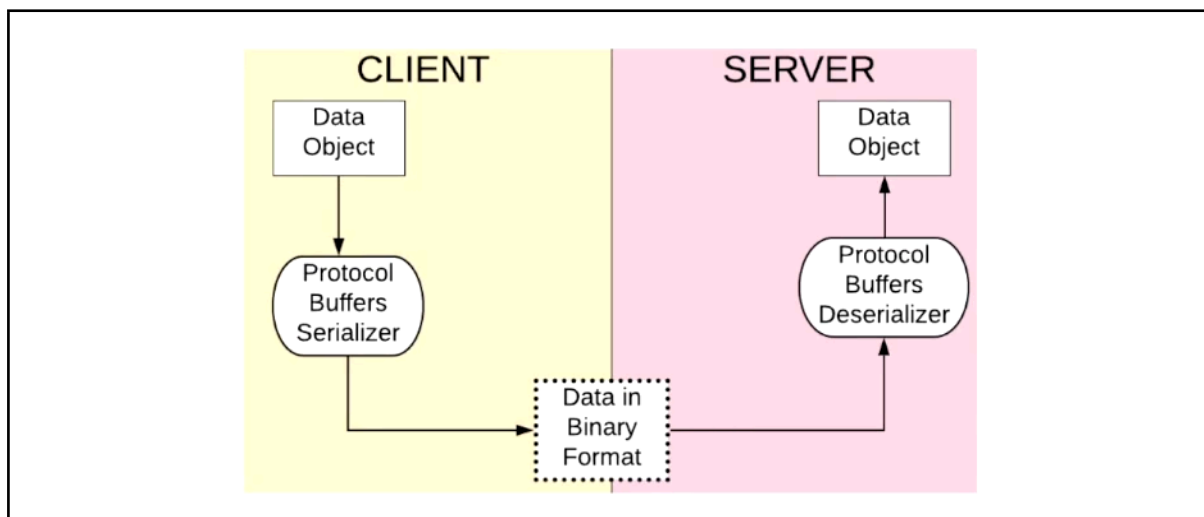
In simple terms, if you need speed, efficiency and microservices communication, gRPC is a great fit. But if you want flexibility and optimized data fetching, GraphQL is the way to go.

gRPC - How it Works?



gRPC is designed for high-performance communication, especially in microservices architecture. It is built on HTTP/2, which brings several key advantages:

- It allows multiple requests, meaning multiple requests and responses can happen simultaneously over a single connection, unlike REST, which is over HTTP/1, where requests open a new connection or wait in sequence.
- Compression happens in HTTP/2, meaning it can reduce payload size, leading to faster and more efficient data transfer compared to JSON-based APIs.
- It is full-duplex streaming, where both clients and servers can send and receive data continuously in real-time.



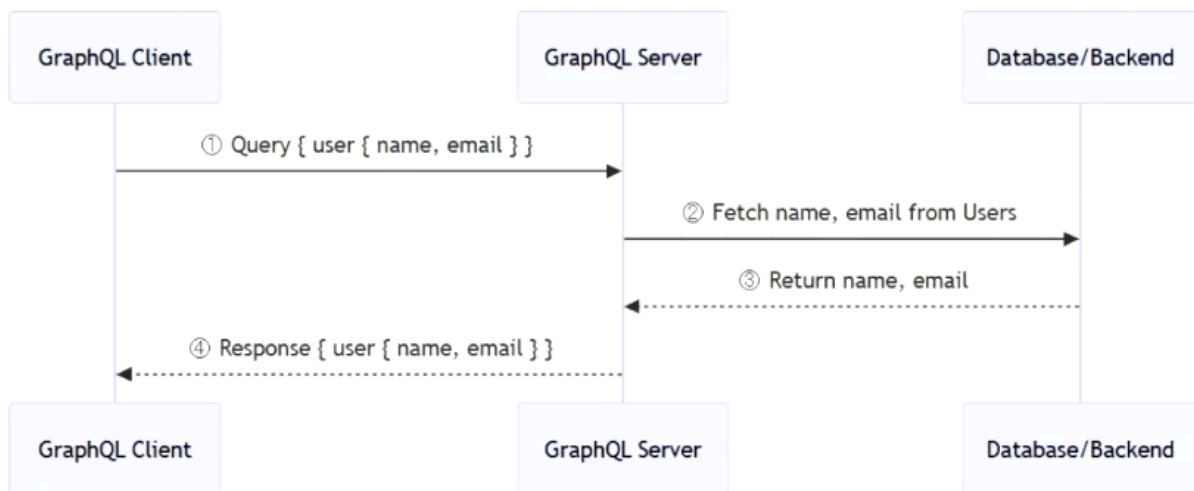
gRPC is a major improvement over REST, which follows a strict request-response protocol. Lastly, gRPC uses protocol buffers (protobuf), instead of JSON for serialization. These protobuf messages are much smaller and faster and efficient, which makes gRPC significantly faster than traditional REST APIs.

gRPC - Use Cases & When to Use

- **Microservices:** Microservices architecture benefits greatly from gRPC because it enables fast, efficient communication between services. Since microservices often exchange large amounts of data across multiple instances, gRPC's low latency and efficient serialisation makes it a perfect fit there.
- **Real-time streaming:** With gRPC's full-duplex bidirectional communication, both the client and the server can send and receive data continuously, which makes it ideal for video streaming, real-time analytics, and financial trading platforms.
- **IoT & Low-bandwidth applications:** Since gRPC uses protocol buffers instead of JSON, it significantly reduces payload size, making communication faster and more efficient, even in the constrained environments.
- **Multi-language ecosystems:** gRPC is a great choice for multi-language ecosystems. It supports auto-generated client and server code for languages like Go, Java, Python, and many more, allowing seamless communication across different tech stacks.

While gRPC is powerful, it may not always be the best choice, especially for public-facing APIs, where REST or GraphQL might be more suitable. It is usually a best fit for communication within your services. If you have a micro-services architecture, it is better to use gRPC for communication between these services.

GraphQL - How does it Works?



Unlike REST, which typically has multiple endpoints for different resources, GraphQL operates with a single endpoint, where clients can specify exactly what data they need, and this will eliminate the issue of overfetching and underfetching seen in the REST API.



At the core of GraphQL, there is the GraphQL schema, which defines the type of data available and how they relate to each other. This schema acts as the contract between the client and server, ensuring that the requested data is structured and validated. Then the client sends queries to the GraphQL server, asking for specific fields, and instead of returning a fixed response, the GraphQL server dynamically resolves each field and assembles based on client's request. This means that the client controls the response shape and thus reduces the unnecessary data transfer.

GraphQL - Use Cases & When to Use

- **Frontend Optimization:** Frontend applications get optimized in the GraphQL usage. Clients can request exactly what they need, avoiding unnecessary data transfer. This is especially beneficial for modern UI frameworks where the different components may need different sets of data.
- **Reducing API Requests:** In REST fetching related data often requires multiple API calls, but with GraphQL, a single query can request data from multiple related entities and thus reducing network overload and ultimately improving the performance.
- **Mobile and Web Applications:** GraphQL is highly useful in slow or unstable network conditions. Since clients receive only the data they require, payloads are much smaller, leading to faster responses and a smoother user experience.
- **Aggregating Data from Multiple Services:** If the application pulls data from multiple databases or APIs, GraphQL can unify these sources and provide a single structured response to the client, making it much easier to work with the distributed systems.

Summary & Key Takeaways

- gRPC → Best for microservices, real-time streaming, high-performance APIs.
- GraphQL → Best for frontend-driven APIs, flexible data fetching, and reducing over-fetching.
- Choose based on use case – No one-size-fits-all!
- Be ready to justify your API choices in system design interviews!
- What's next:
 - Summary & Practical Applications of protocols

Interview Questions on API Protocols

- How would you compare REST, gRPC, and GraphQL?
- When would you use gRPC over REST?
- What are the trade-offs of using GraphQL in a large-scale system?
- How does gRPC handle authentication & security?
- How do you scale GraphQL APIs efficiently?